Semana 04 | Módulo 2: IA, Objetos e o Lago de Dados

O Paradigma Vetorial no SwiftTrack IoT

Arquiteturas de Dados, IA e Busca Semântica na Logística



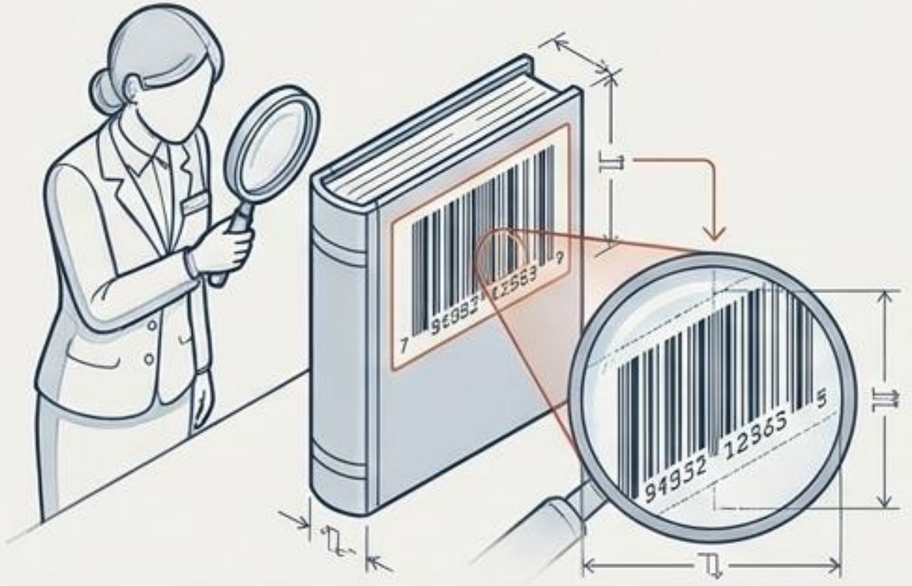
A Quebra do Modelo Tradicional



Sistemas relacionais dependentes de índices exatos ou análise léxica falham estruturalmente ao processar erros humanos de digitação, ambiguidades e abreviações em larga escala.

O Significado Supera a Sintaxe

O Código Exato (Léxica)



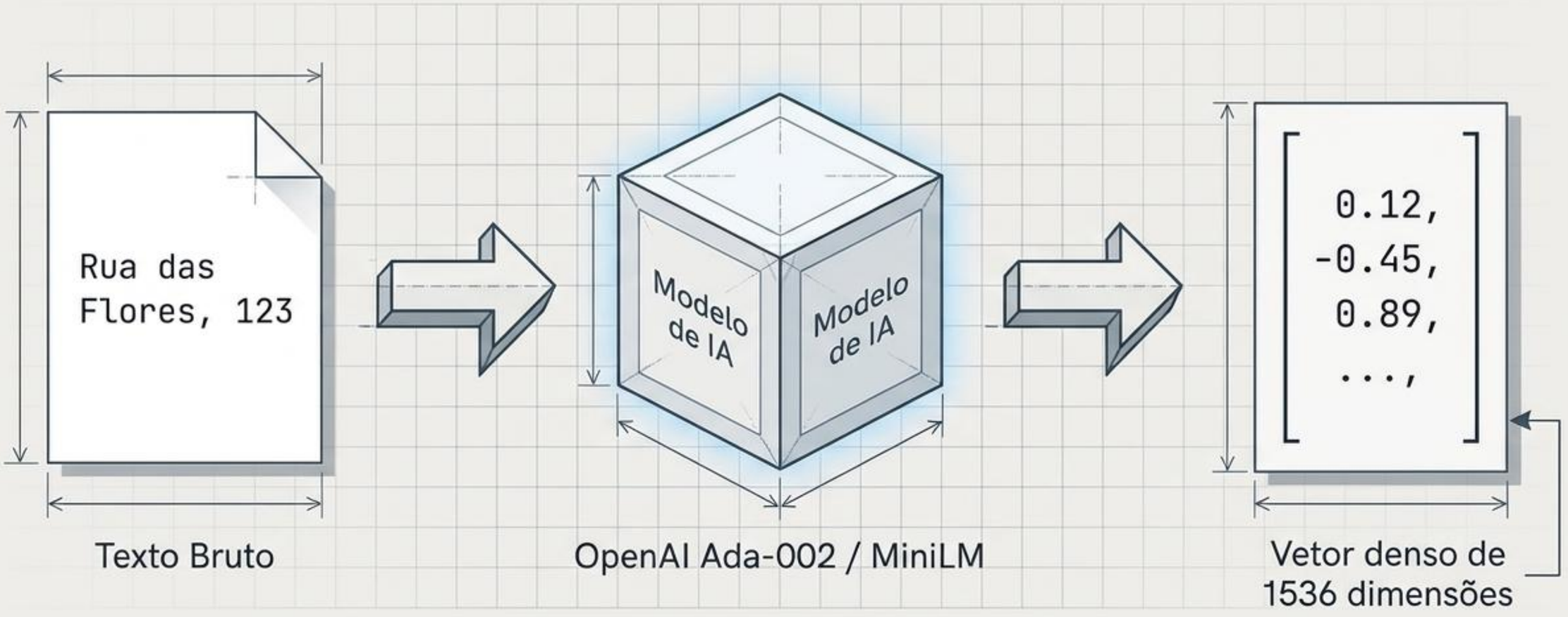
Bancos de dados clássicos exigem um match perfeito de caracteres. Se errar uma letra, a busca falha.

O Contexto (Semântica)



A IA busca pelo conceito subjacente. Ela entende que 'Avenue Paulist' compartilha a mesma vizinhança semântica de 'Av. Paulista'.

Onde o Significado Ganha Coordenadas



Embeddings traduzem a linguagem humana em um espaço de coordenadas de alta dimensionalidade que a máquina pode navegar matematicamente.

A Matemática da Similaridade

	Distância Euclidiana (L2)	Produto Interno	Similaridade de Cosseno
Conceito	Distância em linha reta geométrica	Projeção vetorial (magnitude)	Ângulo direcional entre vetores
Operador `pgvector`	`<->`	`<#>`	**`<=>`**
Uso Ideal	Dados espaciais ou imagens brutas	Sistemas de recomendação diretos	Textos normalizados (Nosso foco)

Para endereços logísticos onde o tamanho do texto varia mas a direção semântica importa, o Cosseno é o padrão-ouro matemático.

pgvector: A Extensão que Muda o Jogo

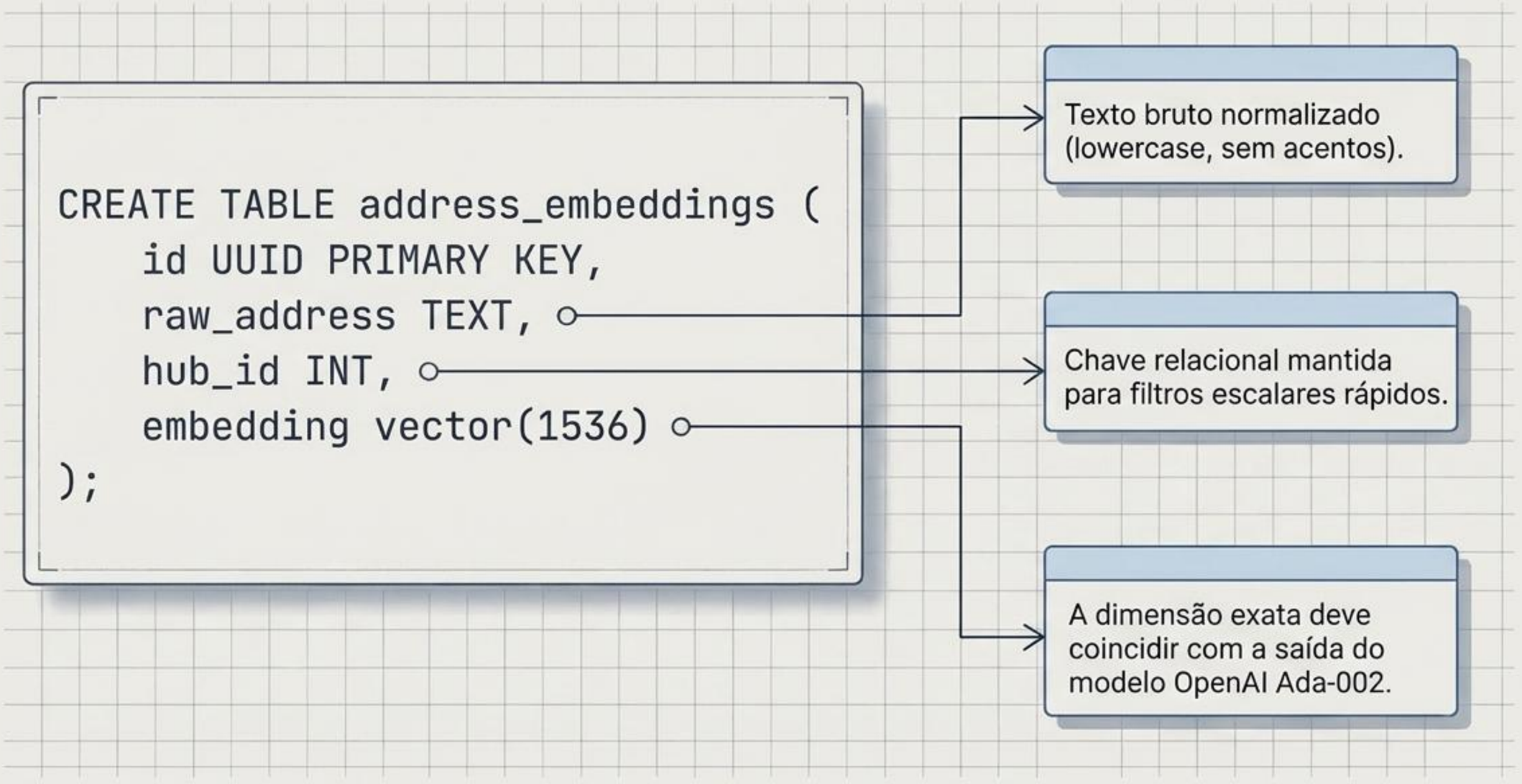



```
CREATE EXTENSION vector;
```

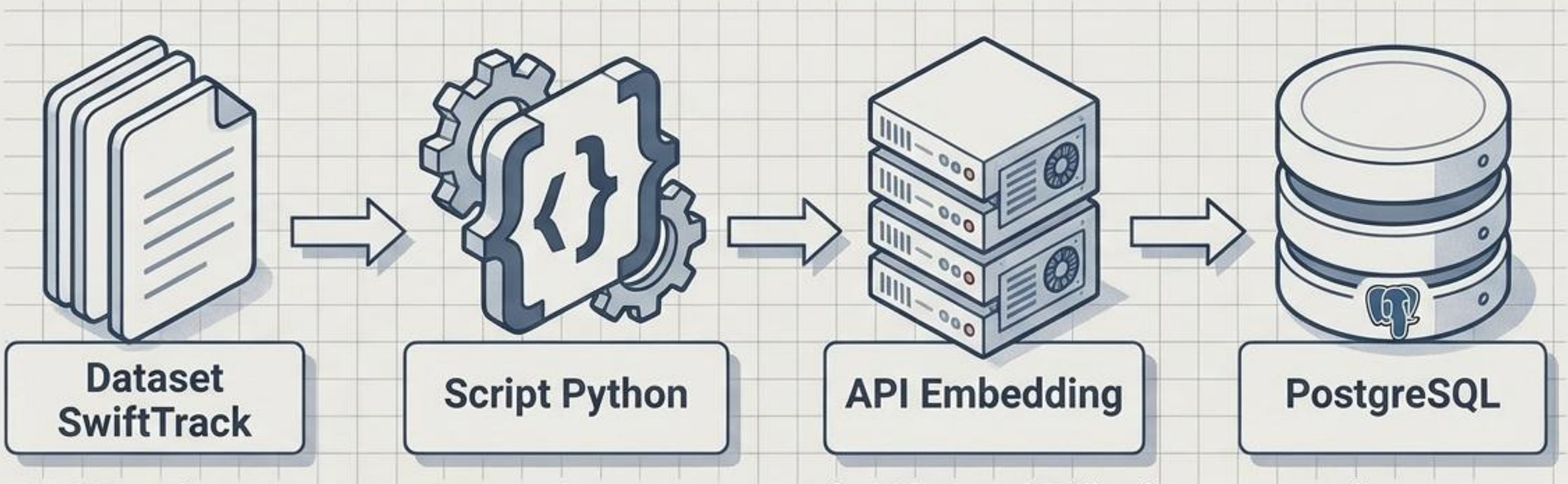
Unificação ACID: Manter dados relacionais (IDs de motorista) e vetoriais na mesma transação reduz drasticamente a complexidade arquitetural.

Casos Avançados: A mesma estrutura pode cruzar embeddings de incidentes de rota para detectar anomalias.

Modelando o Espaço Vetorial



O Pipeline de Ingestão



Atenção ao Custo: Processamento em lote (batch) é mandatório para a carga inicial devido à latência de rede.

Fallback Local: O uso de modelos leves (all-MiniLM-L6) é vital caso haja restrição de hardware ou cota de API.

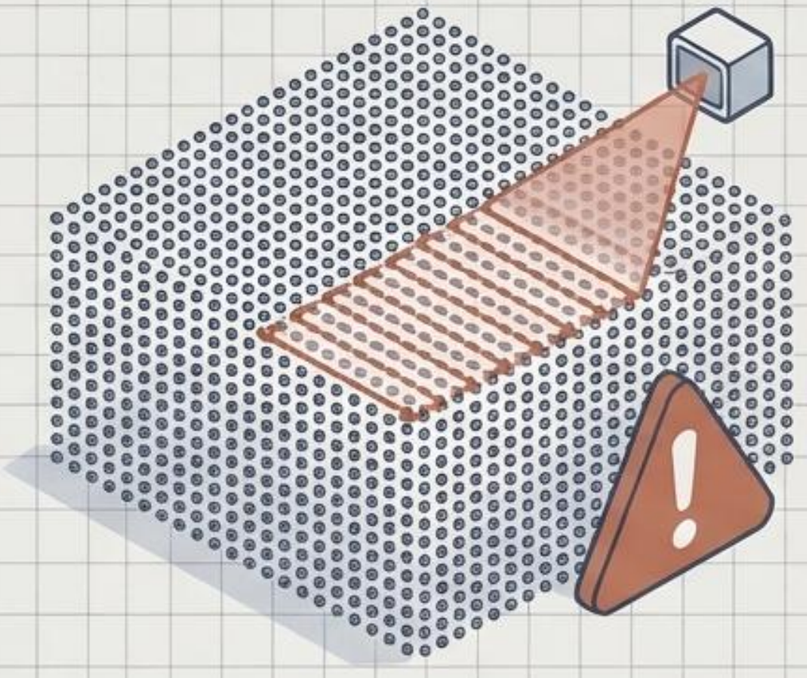
A Resolução K-NN (K-Nearest Neighbors)

```
SELECT raw_address, embedding <=> '[vetor_do_input_errado]' AS distance
FROM address_embeddings
ORDER BY embedding <=> '[vetor_do_input_errado]'
LIMIT 5;
```

A resolução do SAC. Esta query compara o vetor do input com erro de digitação contra todo o banco de dados e retorna os 5 **vizinhos mais próximos** matematicamente, **ignorando** completamente as regras rígidas de ortografia.

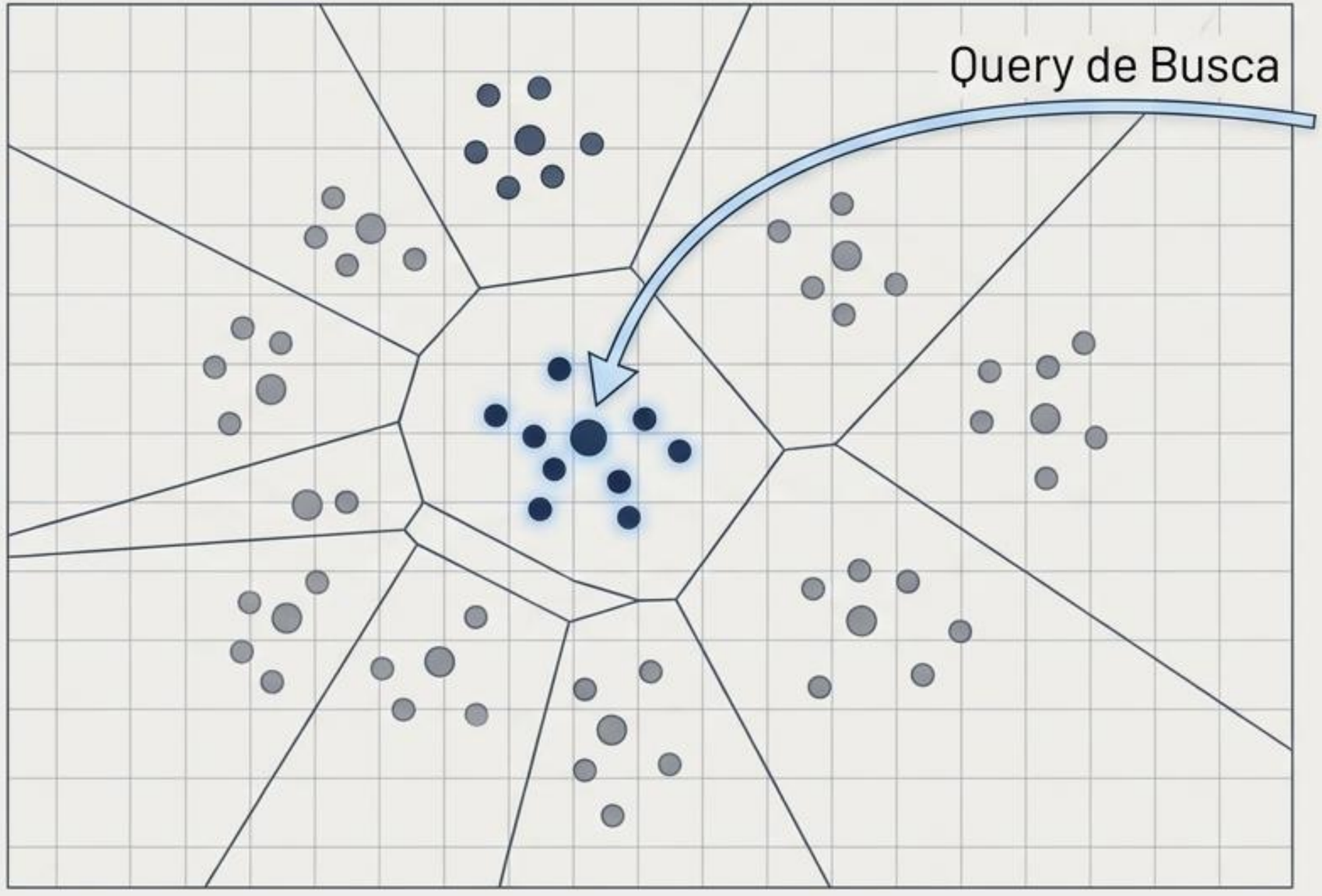
O Gargalo Invisível: Sequential Scan

```
-> Seq Scan on address_embeddings
   Filter: (embedding <=> '[...]'::vector)
   Rows Removed by Filter: 0
   Execution Time: 450.23 ms (WARNING)
```



A **armadilha do K-NN exato:** o motor do banco precisa calcular o cálculo trigonométrico contra todas as linhas da tabela. Perfeito na precisão, insustentável na escala de milhões de rotas reais.

A Solução: O Índice IVFFlat



Inverted File with Flat Compression agrupa vetores em clusters (K-means).

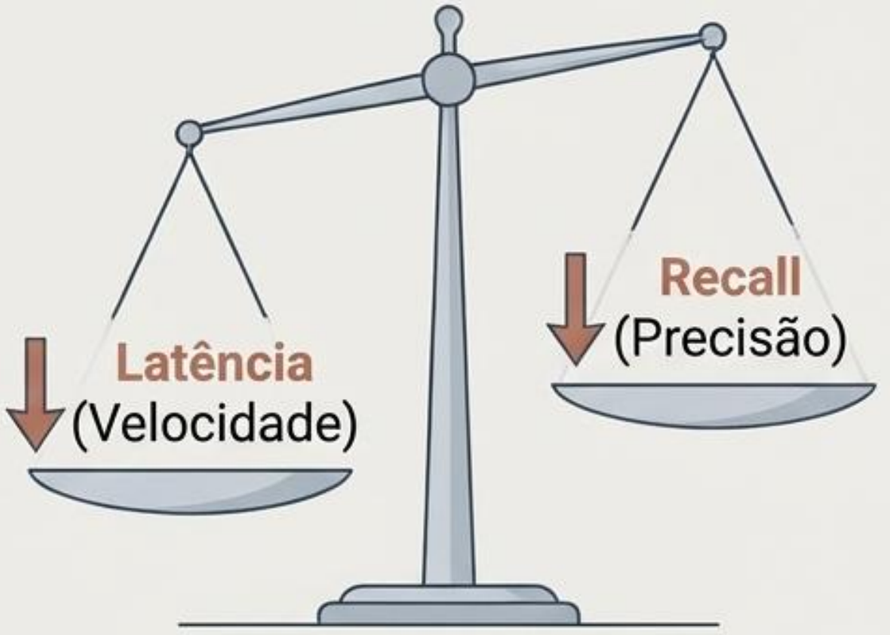
A busca direciona-se apenas ao cluster mais próximo, evitando varrer a tabela inteira.

O resultado passa de exato para aproximado, mas com **ganhos massivos de velocidade**.

Implementação e Sintonia Fina

```
CREATE INDEX ON address_embeddings
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

⚠ **Regra de Ouro:** O índice deve ser criado após a tabela estar populada para que o algoritmo identifique os centros de cluster corretos.



O parâmetro **'lists'** dita o trade-off arquitetural. Aumentar a segmentação acelera a busca exponencialmente, ao custo de uma ligeira queda na precisão estatística.

Onde a Mágica Acontece no Mundo Real?



Edge App
(Motorista)

Envia texto bruto do GPS.



Backend
(Python API)

Gera o embedding em tempo real.



PostgreSQL
(pgvector)

Executa a busca KNN e devolve a sugestão.

Estratégia: A vetorização do endereço alvo ocorre em lote no banco, enquanto o input momentâneo do motorista é vetorizado dinamicamente via API no backend.

Entregáveis da Semana (O Desafio)

1. Código SQL

Subir `week4_deliverable.sql` contendo a modelagem da tabela, criação do índice **IVFFlat** e a query KNN de resolução de erro.

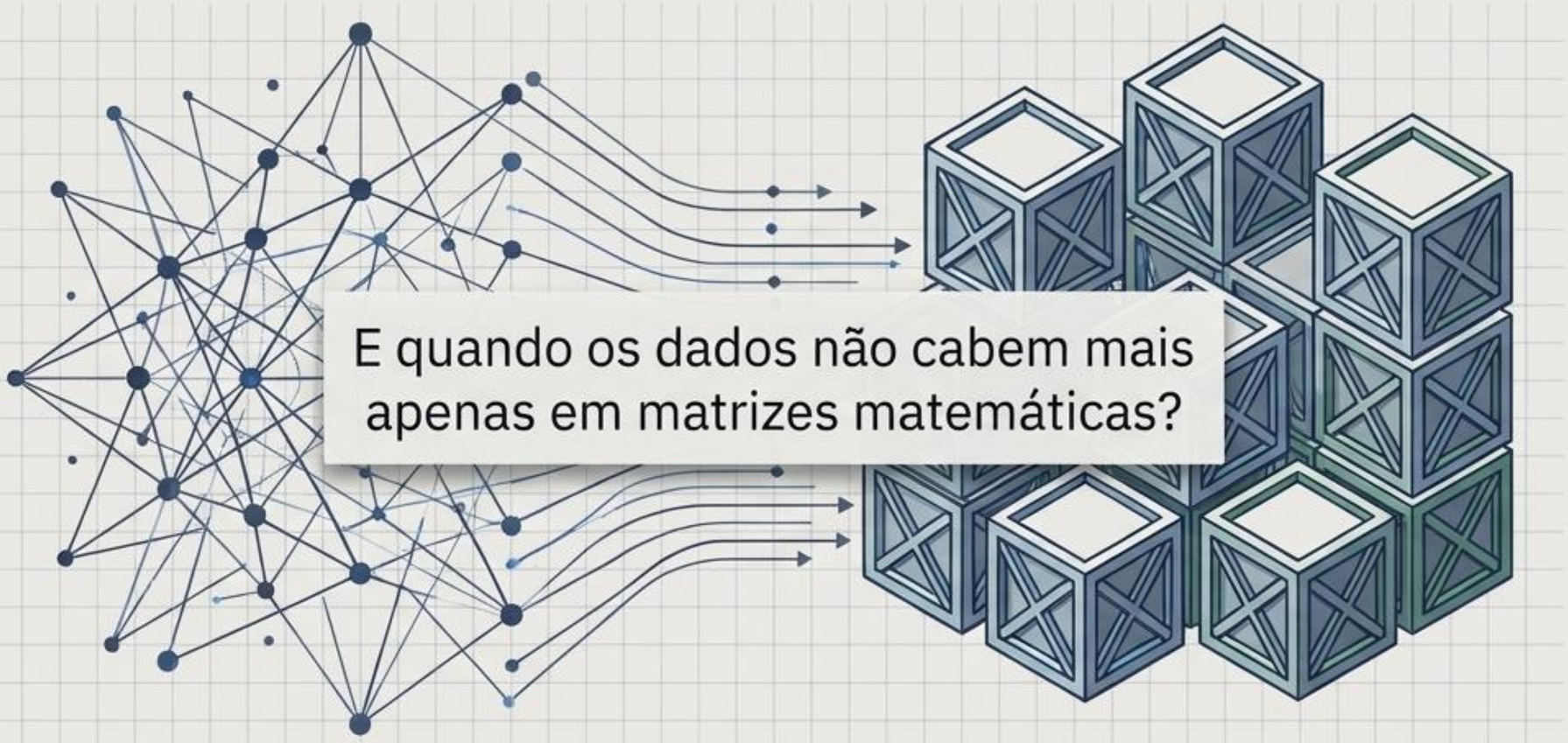
2. Documentação de Decisão

Um `README.md` justificando a arquitetura escolhida (onde ocorre a geração do embedding: Edge, Backend ou Data Lake) visando menor latência.

3. Boas Práticas Aplicadas

Demonstração clara de normalização de texto bruto (`lowercase`, **remoção de acentos**) antes da fase de vetorização algorítmica.

Além dos Vetores: O Lago de Dados



Na Semana 5, conectaremos o **PostgreSQL** ao MinIO. Expandiremos a arquitetura para armazenar dados não estruturados pesados, como as fotos de Comprovantes de Entrega (POD), fundando o nosso Data Lakehouse.